



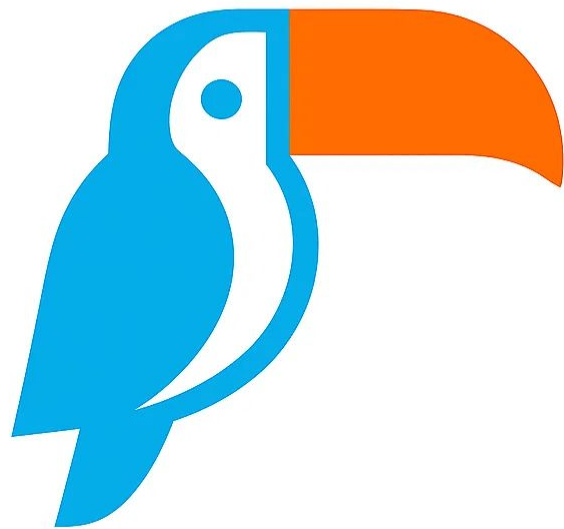
GeneRAG

Desarrollo y evaluación de un sistema basado en RAG de consultas ciudadanas sobre ayudas gubernamentales

Autor: Pablo Valenzuela Álvarez
Director: David Griol Barres

Septiembre, 2025

MÁSTER UNIVERSITARIO EN INGENIERÍA INFORMÁTICA



GeneRAG



Tabla de contenidos

1. Introducción

**3. Desarrollo
del proyecto**

**5. Conclusiones y
Líneas Futuras**

2. Estado del Arte

4. Evaluación

1. Introducción



1. Introducción

Dominio de la aplicación

BOE + DANA

- ✓ Fuente oficial
- ✓ Información veraz
- ✗ Documentos densos
- ✗ Difícil comprensión
- ✗ Acceso complicado

Pasarela provisional en la zona del barranco del Poyo en Picanya (Valencia), el 27 de noviembre.

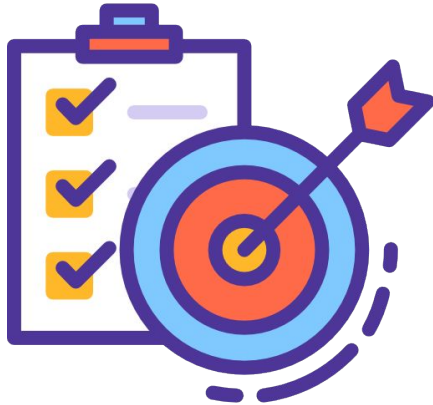


<https://www.rtve.es/noticias/20241129/dana-valencia-graficos-mapas-imagenes/16345347.shtml>



1. Introducción

Objetivos



- 1 Desarrollo de un sistema RAG aplicado a este dominio
- 2 Evaluación detallada del sistema desarrollado



1. Introducción

Plataforma



Codificación difícil

Tiempo de entreno

Rasa Pro

- Licencia de Uso



Documentación

Integraciones

- Ollama
- OpenAI y MistralAI
- ChromaDB

Python



1. Introducción



Apps de ayuda

Ajuda Dana

Voluntariado

TuCocheDana

Justicio

BOE + RAG

Pregunta a Justicio

Respuestas legales fundamentadas

ABIERTO · LIBRE · GRATUITO





1. Introducción

Vertientes

Social



Práctica



Técnica



2. Estado del Arte



2. Estado del Arte

RAG

Retrieval-Augmented-Generation o Generación Aumentada por Recuperación

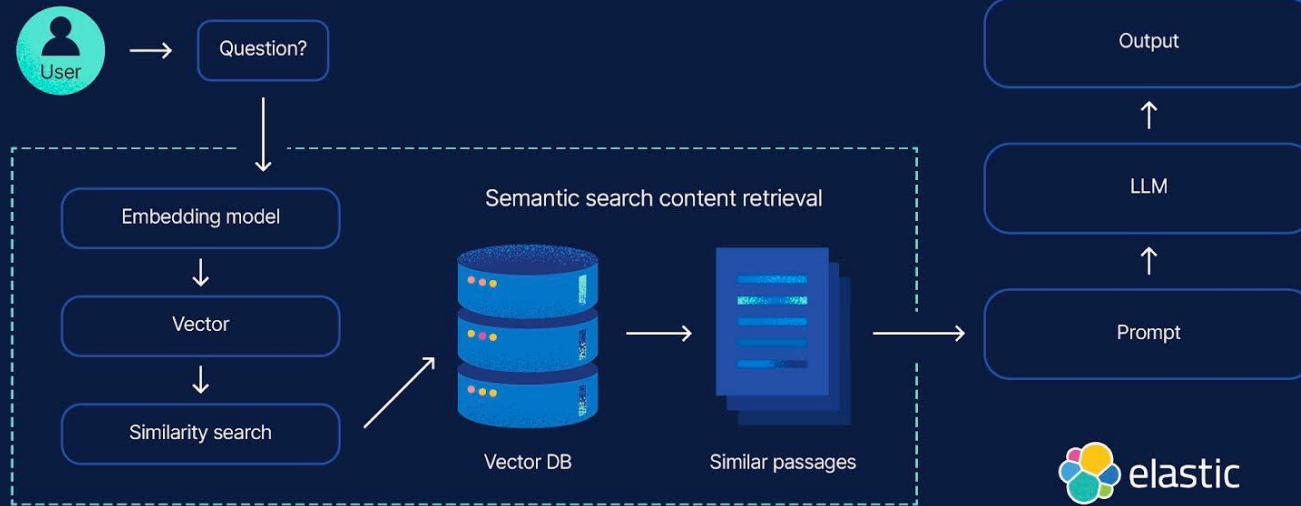
“Enfoque que enriquece la respuesta ofrecida por un **LLM** sirviéndose de un modelo de recuperación ayudado por un **embedding** enfocado a un dominio en específico”





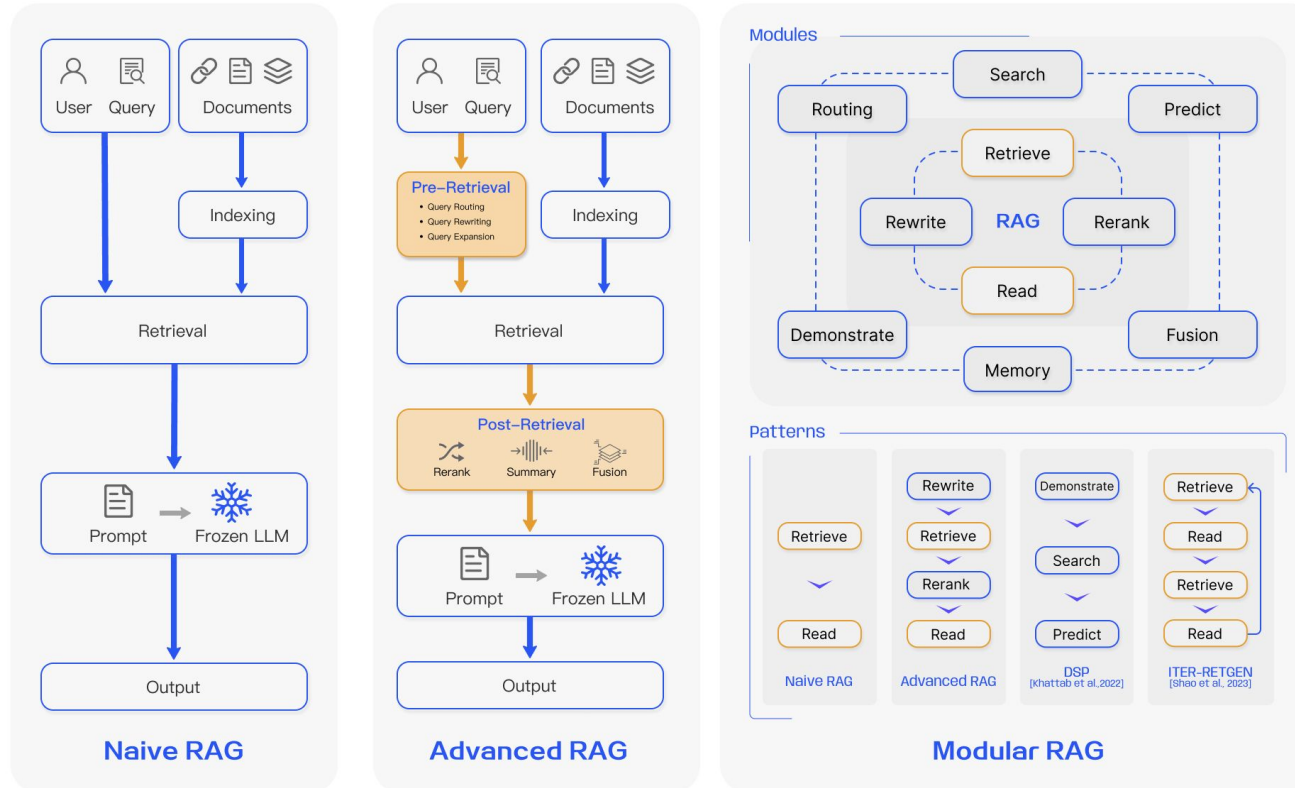
2. Estado del Arte

Retrieval augmented generation (RAG) in action





2. Estado del Arte

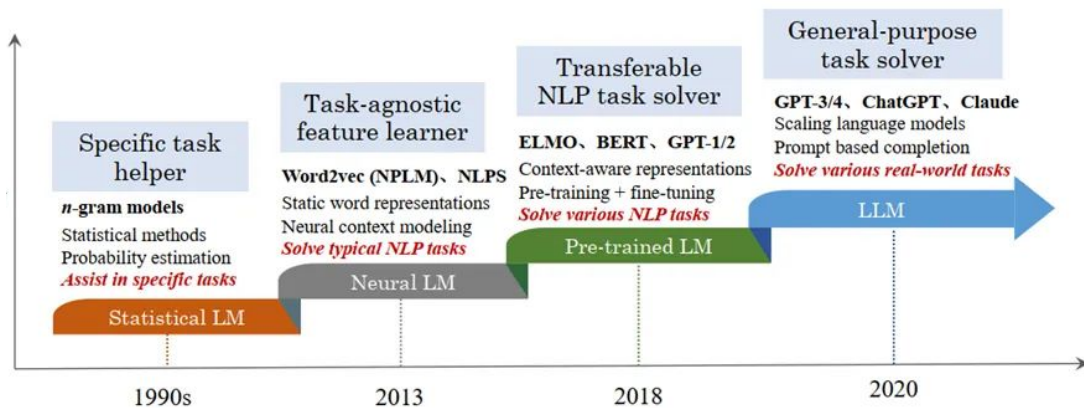




2. Estado del Arte

LLM

Large Language Models o
Grandes modelos de lenguaje



<https://arxiv.org/abs/2303.18223>



2. Estado del Arte

LLM



- Llama 3.2
- 3B
- Ollama 
- Gratuito



- Mistral Small 3.2
- 24B
- API
- Gratuito

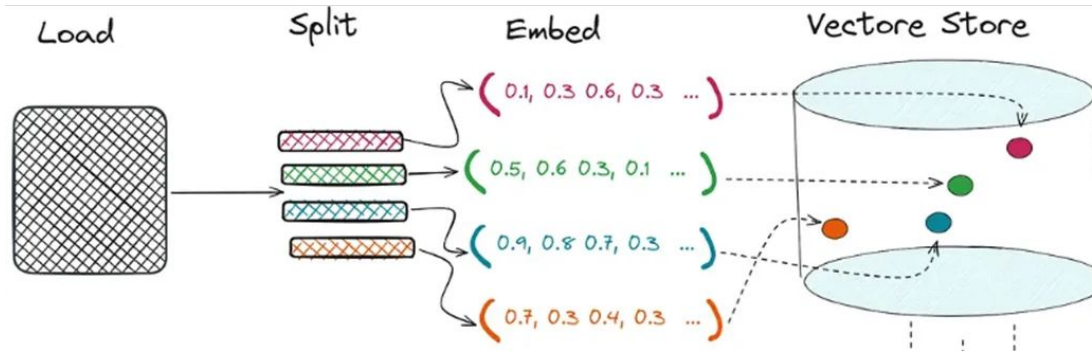
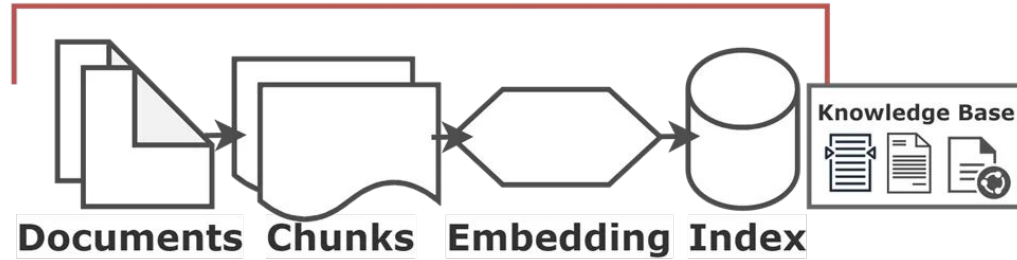


- GPT-4.1
- ???
- API
- De pago



2. Estado del Arte

Vector Store y Embedding



<https://mehmetozkaya.medium.com/the-rag-architecture-ingestion-and-retrieval-with-embeddings-and-vector-search-9bb86adb0ff5>



2. Estado del Arte

Vector Store



Embedding

NOMIC



text-embedding-3-small

3. Desarrollo del proyecto



3. Desarrollo del proyecto

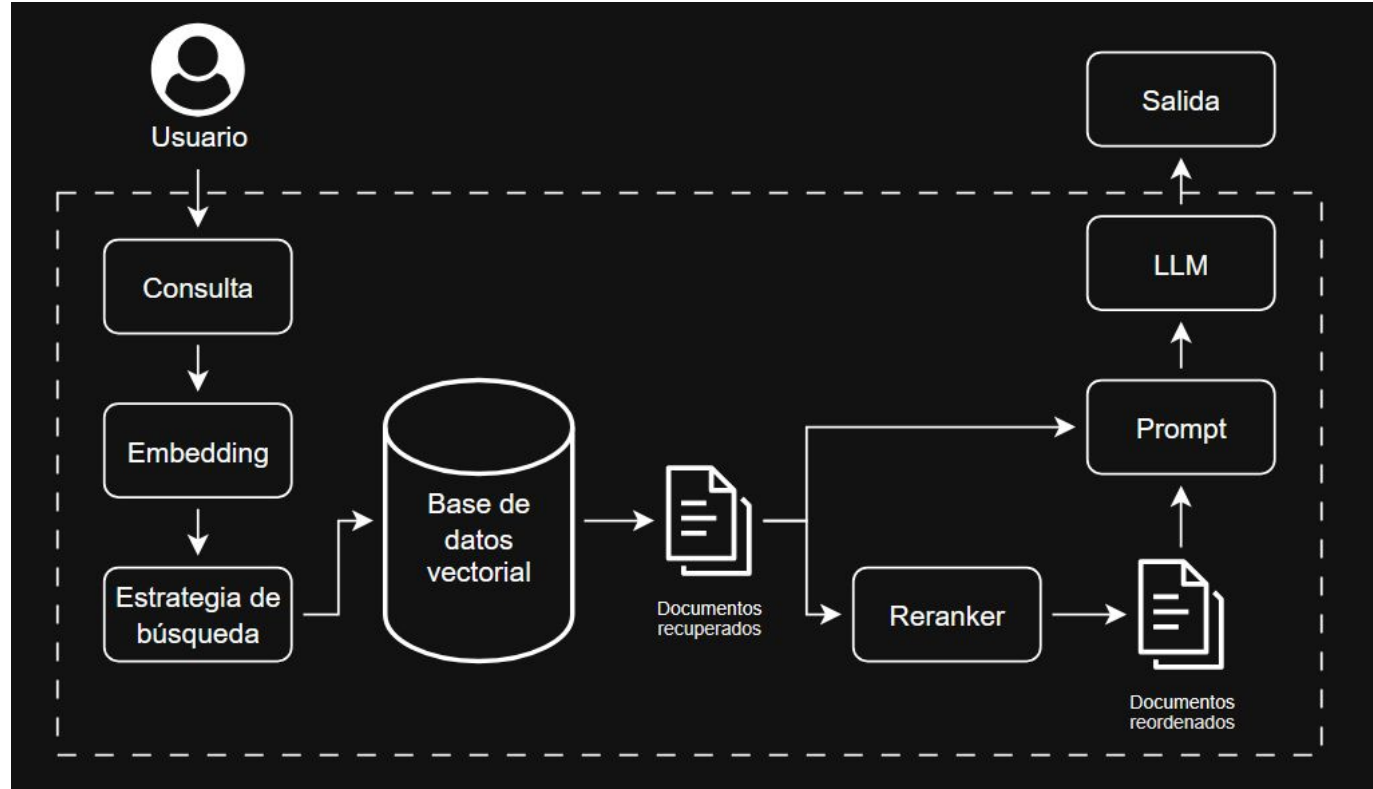
Puntos

- Diseño
- Metodología Scrum
- CI/CD
- Constructor Chatbot
- Interfaz Streamlit
- Recuperación de información
- Generación de texto
- Capturas



3. Desarrollo del proyecto

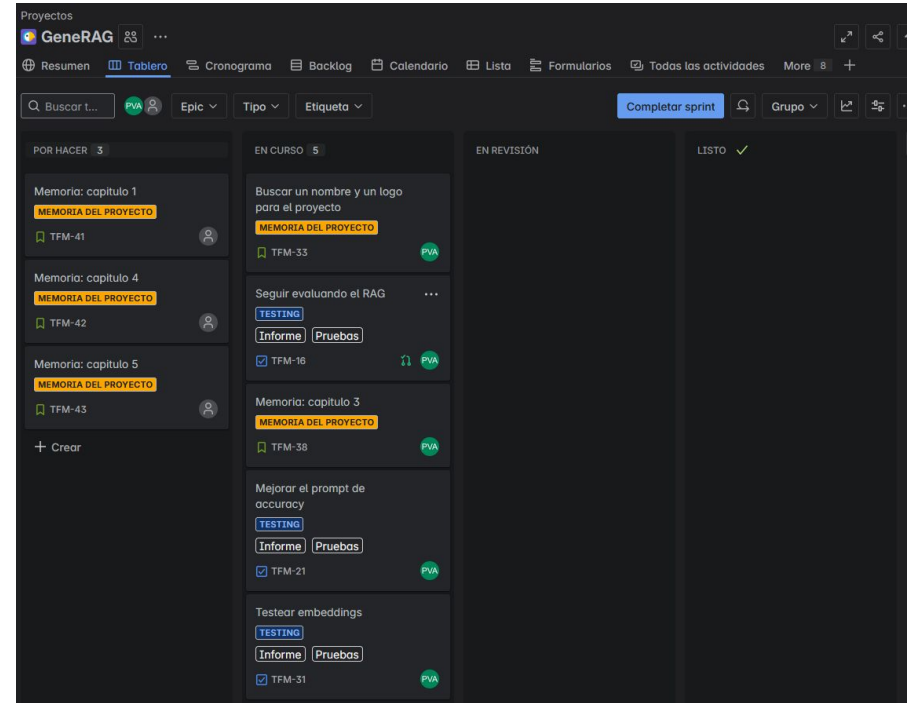
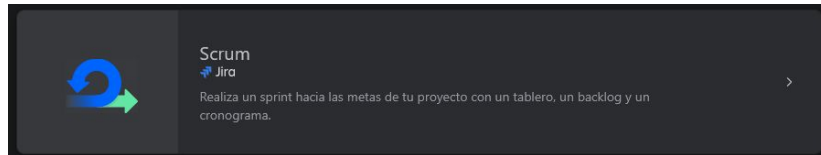
Diseño





3. Desarrollo del proyecto

Metodología Scrum

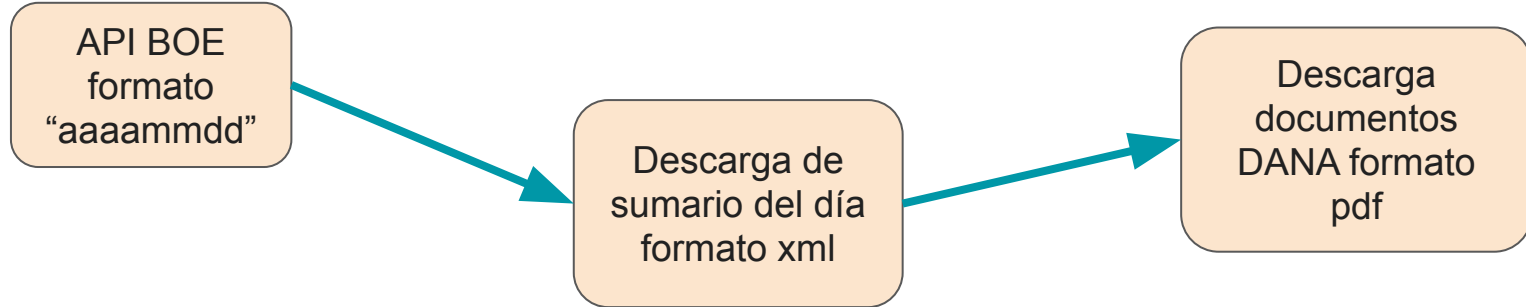




3. Desarrollo del proyecto

CI/CD - Github Actions

Obtención de la información del BOE y filtrado de documentos



<https://www.boe.es/datosabiertos/api/api.php>



3. Desarrollo del proyecto

CI/CD - Github Actions

on:
schedule:
- cron "0 9 * * 1-6"

runs on: ubuntu

steps:
- checkout
- python setup
- script descarga sumario
- script descarga documentos
- commit



3. Desarrollo del proyecto

Chatbot: constructor

```
44 class Chatbot:
45
46     ### Constructor ###
47     def __init__(self,
48         language_model: str = "mistral-small-latest", num_ctx: int = 4096,
49         chunk_size: int = 1024, chunk_overlap: int = 100,
50         embedding_model: str = EMBEDDING.NOMIC.value,
51         search_type: str = "similarity", k: int = 5,
52         chroma_directory: str = "chroma",
53         docs_directory: str = "my_data",
54         reranker_model: str = "BAAI/bge-reranker-v2-m3",
55         top_n: int = 3,
56         prompt_template: str = default_prompt
57     ):
```




3. Desarrollo del proyecto

Chatbot: constructor

LLMs:

- Mistral → ChatMistralAI 
- GPT-4.1 → ChatOpenAI 
- Llama3.2 → ChatOllama

Embeddings:

- OpenAIEmbeddings 
- OllamaEmbeddings

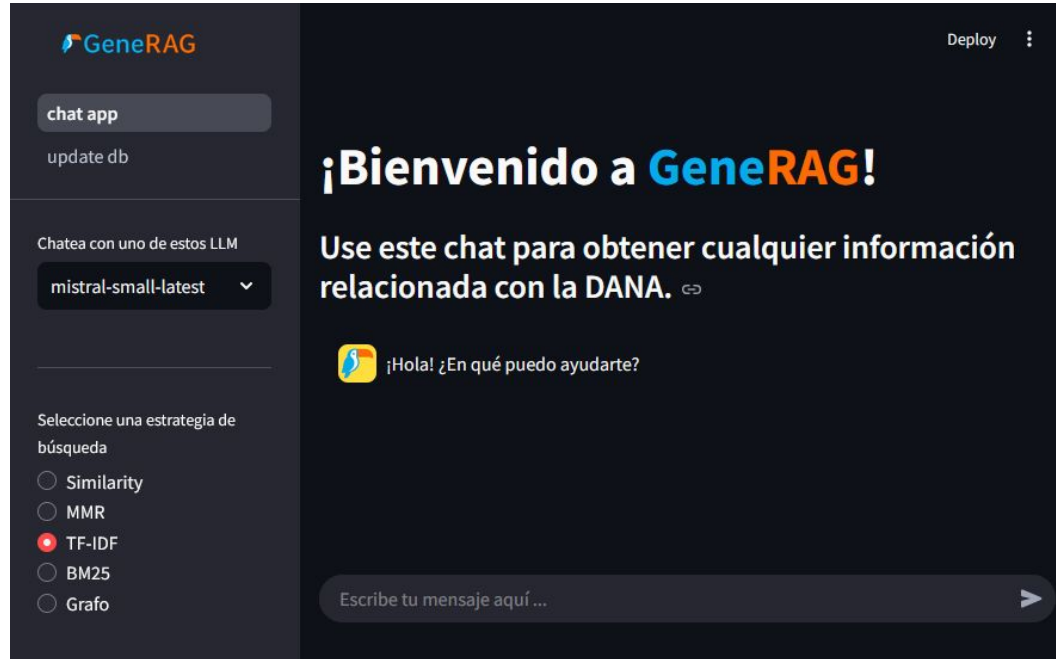
Search Strategy:

- Similarity → Default
- MMR → Default
- TF-IDF → TFIDFRetriever
- BM25 → BM25Retriever
- Graph → GraphRetriever



3. Desarrollo del proyecto

Interfaz Streamlit: chatbot





3. Desarrollo del proyecto

Interfaz Streamlit: carga y actualización

GeneRAG <

chat app

update db

Actualizar la base de datos

Número de documentos existentes en la base de datos: 2638

Carga desde directorio

Selección y Cargar

Carga desde fichero/s

Drag and drop files here
Limit 200MB per file • PDF

Browse files

Deploy



3. Desarrollo del proyecto

Interfaz Streamlit: carga y actualización

Actualizando base de datos



Realizando el proceso, espere ... (3 minutes, 48.5 seconds)



Añadiendo nuevos documentos: 4179

Actualizando base de datos



Añadiendo nuevos documentos: 4179



Tiempo empleado: 806.92 segundos

¡Proceso finalizado con éxito!

Actualizando base de datos



Añadiendo nuevos documentos: 2876



Tiempo empleado: 40.33 segundos

¡Proceso finalizado con éxito!



3. Desarrollo del proyecto

Recuperación de información

```
# retriever = chatbot.get_retriever()          # normal
retriever = chatbot.get_compression_retriever() # reranker

results = retriever.invoke(question)
context = "\n\n--\n\n".join([doc.page_content for doc in results])
metadata = [
    {
        "author": doc.metadata.get("author", None),
        "creator": doc.metadata.get("creator", None),
        "id": doc.metadata.get("id", None),
        "keywords": doc.metadata.get("keywords", None),
        "source": doc.metadata.get("source", None),
        "subject": doc.metadata.get("subject", None),
        "title": doc.metadata.get("title", None)
    }
    for doc in results
]

sources_set = {item["id"] for item in metadata if item.get("id")}
sources = "---\n\n**Referencias**:\n\n" + "\n\n".join(f"\t{src}" for src in sources_set)
```



3. Desarrollo del proyecto

Generación de texto

```
response_text = st.write_stream(  
    chatbot.answer_query(question, context)  
)  
st.write(sources)  
  
return response_text + "\n\n" + sources
```

```
def answer_query(self, query: str, context: str):  
    query_chain = (  
        {"context": self.retriever, "question": RunnablePassthrough(), "context": RunnablePassthrough()}  
        | self.prompt_template  
        | self.language_model  
        | StrOutputParser()  
    )  
    return query_chain.stream({"question": query, "context": context})
```



3. Desarrollo del proyecto

Diseño del prompt

```
default_prompt = """
Eres un asistente que responde preguntas usando SOLO y ÚNICAMENTE el contexto proporcionado.
Si la respuesta NO está en contexto, simplemente di que no lo sabes y no respondas o te inventes la respuesta.
Tus respuestas DEBEN ser detalladas y estar bien estructuradas, organizando la información en párrafos y listas si es necesario.
Tus respuestas DEBEN de ser respondidas con las partes de información sacadas del contexto.
Tus respuestas NO DEBEN mencionar expresiones como "de acuerdo a la información" o "según el contexto".

Si el usuario pide información sobre ayudas, estas también pueden ser subvenciones.
Si el usuario pide información sobre subvenciones, estas también pueden ser ayudas.
Todas las preguntas realizadas por el usuario están relacionadas con la DANA (Depresión Aislada en Niveles Altos) sucedida a finales de 2024.

Aquí tienes la pregunta y el contexto:|

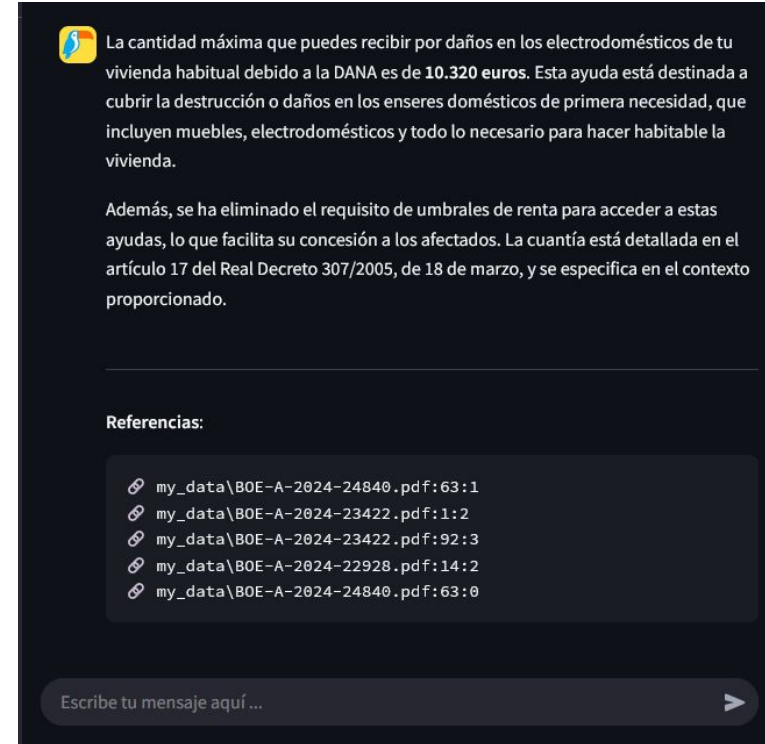
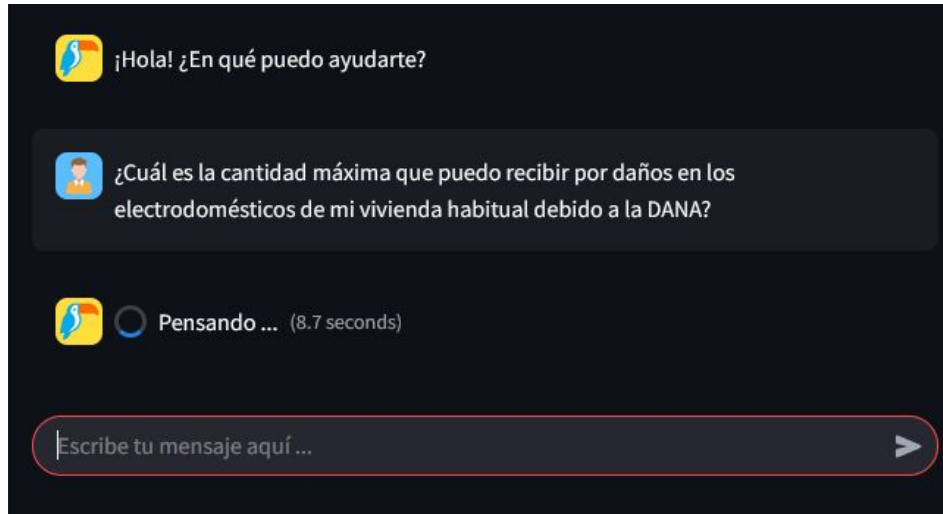
Pregunta: {question}\n
Contexto: {context}\n

```



3. Desarrollo del proyecto

Capturas



4. Evaluación



4. Evaluación

Puntos

- Agentes críticos
- Generación de corpus pregunta-respuesta
- Evaluaciones



4. Evaluación

Agentes críticos [1-5] (LLM as a Judge)

- Accuracy
- Faithfulness
- Groundedness
- Relevance



<https://towardsdatascience.com/llm-as-a-judge-a-practical-guide/>



4. Evaluación

Generación corpus de preguntas-respuestas (~1h)

```
QA_generation_prompt = """
Tu tarea es generar una ÚNICA pregunta con su respectiva respuesta basándote únicamente en el contexto aportado.
Tus preguntas deben estar relacionadas con la DANA (Depresión Aislada en Niveles Altos) ocurrida a finales de 2024.
Las preguntas generadas deben tener un formato de preguntas frecuentes (FAQ).
Debes formular tus preguntas imitando el estilo que usaría una persona afectada por la catástrofe buscando información.
Tus preguntas deben poder ser respondidas usando una pieza específica del contexto.
No debes mencionar cosas como "de acuerdo a la información" o "según el contexto".
Escribe en ESPAÑOL.
---
Realiza tu respuestas como sigue, manteniendo un formato separado por comas (csv):

"tu pregunta","tu respuesta"

---
Contexto: {context}\n
Output:::"""
```



4. Evaluación

Generación corpus de preguntas-respuestas

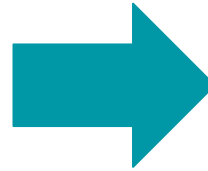
Formato:

? Pregunta

... Respuesta

Contexto

Recurso



300 preguntas



4. Evaluación

Mistral Small 3.2: docs 512 (~3h)

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
bge-m3	tfidf rr	3.46	4.79	4.71 *	4.51 *
jina	tfidf rr	3.52 *	4.77	4.66	4.5
nomic	tfidf rr	3.5	4.8	4.68	4.49
snowflake	tfidf rr	3.51	4.78	4.67	4.5



4. Evaluación

Mistral Small 3.2: docs 1024 (~3h)

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
bge-m3	tfidf rr	3.77	4.91	4.85	4.75
jina	tfidf rr	3.84 *	4.94	4.92 *	4.81 *
nomic	tfidf rr	3.72	4.9	4.82	4.75
snowflake	similarity rr	3.69	4.91	4.81	4.64

* jina + bm25_rr → faithfulness = 4.95



4. Evaluación

Mistral Small 3.2: docs 1024 + modificación del prompt 🏠 (~3h)

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
bge-m3	tfidf rr	3.72	4.95	4.4	4.82 *
jina	tfidf rr	3.66	4.92	4.3	4.73
nomic	tfidf rr	3.67	4.91	4.3	4.75
snowflake	tfidf rr	3.65	4.92	4.27	4.75



4. Evaluación

Llama 3.2: docs 1024 ∞ (~3h20m *)

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
jina	similarity rr	3.08	4.17	3.75	4.25
jina	grafo	2.94	4.29	3.18	4.18
jina	similarity	2.76	3.94	3.41	4.24
jina	tfidf rr	2.5	4.5	3.33	4.67



4. Evaluación

GPT-4.1: docs 1024 (~40m *)

Embedding	Estrategia	Accuracy	Faithfulness	Groundedness	Relevance
te3s	tfidf	4.32 *	4.95 *	4.91	4.68
te3s	tfidf rr	4.27	4.92	4.96 *	4.75
te3s	similarity rr	3.96	4.89	4.88	4.41
te3s	similarity	3.95	4.88	4.89	4.49



4. Evaluación

Conclusiones de los resultados



Contexto



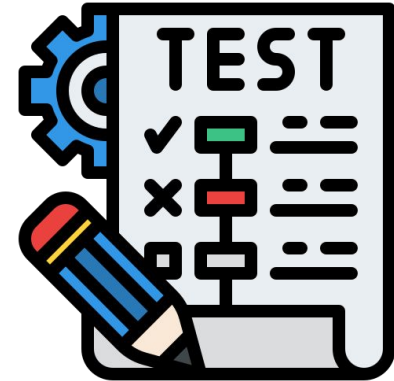
Prompt



Embedding



LLM



5. Conclusiones y Líneas Futuras



5. Conclusiones y Líneas Futuras

- ◆ Auge LLM
- ◆ Langchain
- ◆ Streamlit
- ◆ Resultados

- ◆ Personalización
- ◆ Contenerización

- ◆ Facilitar acceso a información veraz y comprensible en momentos de crisis
- ◆ IA para mejorar el futuro



Turno de preguntas

GeneRAG

Desarrollo y evaluación de un sistema basado en RAG de consultas ciudadanas sobre ayudas gubernamentales

Autor: Pablo Valenzuela Álvarez
Director: David Griol Barres

Septiembre, 2025